МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Программирование»

Тема: Типы данных и их внутреннее представление в памяти

Студент гр. 6372 _____ Иващенко А.А.

Санкт-Петербург
2026

1. Цель и содержание работы

Цель работы - изучить внутреннее представление целых и вещественных чисел в памяти компьютера, научиться анализировать отдельные биты значения, а также сравнить особенности хранения целочисленных типов и чисел с плавающей точкой. В ходе работы были рассмотрены коды целых чисел, формат IEEE 754 для типов float и double, особые значения, битовые операции и безопасный способ получения объектного представления через std::memcpy.

2. Представление целых чисел в памяти

Целое число в памяти хранится как последовательность битов фиксированной длины. Для беззнакового типа каждый бит имеет вес, равный соответствующей степени двойки.

Для знаковых целых чисел необходимо кодировать не только модуль, но и знак. Есть несколько вариантов представления числа: прямой, обратный и дополнительный коды. Однако практически во всех современных компьютерах для хранения целых чисел со знаком используется дополнительный код, так как он решает проблему двойного нуля и упрощает арифметические операции.

В дополнительном коде положительные числа имеют обычное двоичное представление. Для получения представления отрицательного числа можно инвертировать все биты модуля и прибавить единицу. Например, для 8-битного представления числа -5:

```
 5  = 00000101
~5  = 11111010
+1  = 11111011   -> представление -5
```

Старший разряд знакового целого числа содержит информацию о знаке: для положительных чисел он равен 0, для отрицательных - 1.

В программе вывод битов целого числа выполняется после преобразования значения в unsigned. Такое преобразование позволяет безопасно применять сдвиги и маски к битовому представлению:

```
void printIntBits(int value, int width = 32) {
    unsigned u = static_cast<unsigned>(value);
    for (int i = width - 1; i >= 0; --i) {
        putchar((u >> i) & 1 ? '1' : '0');
    }
}
```

Выражение (u >> i) & 1 сдвигает нужный разряд в нулевую позицию и оставляет только его значение. Таким образом, биты выводятся от старшего к младшему.

3. Формат IEEE 754 для вещественных чисел

Вещественные числа типов float и double хранятся по принципу числа с плавающей точкой. В распространённом формате IEEE 754 значение разбивается на три логических поля: знак, экспоненту и мантиссу.

Тип	**Всего бит**	**Знак**	**Экспонента**	**Мантисса**
float	32	1	8	23
double	64	1	11	52

3.1. Нормализованные и денормализованные значения

Для нормализованных чисел поле экспоненты не состоит полностью из нулей и не состоит полностью из единиц. Старшая единица мантиссы не хранится явно и считается равной 1. Это называют скрытой или неявной единицей.

Если поле экспоненты состоит из нулей, а мантисса ненулевая, число является денормализованным. В этом случае скрытая единица отсутствует, что позволяет представлять значения, расположенные очень близко к нулю. Минимальное положительное денормализованное значение удобно получать средствами стандартной библиотеки:

```
float denormFloat =
std::numeric_limits<float>::denorm_min();
double denormDouble =
std::numeric_limits<double>::denorm_min();
```

3.2. Особые значения

IEEE 754 предусматривает специальные комбинации битов. При экспоненте, состоящей из единиц, и нулевой мантиссе хранится бесконечность. Если экспонента заполнена единицами, а мантисса ненулевая, кодируется NaN (Not a Number). Бесконечность в программе получалась следующим образом:

```
float infFloat =
std::numeric_limits<float>::infinity();
double infDouble =
std::numeric_limits<double>::infinity();
```

4. Переинтерпретация битов с помощью std::memcpy

Для анализа отдельных полей IEEE 754 необходимо получить не числовое значение после преобразования типа, а точную последовательность битов, уже находящуюся в памяти. В программе для этого используется функция std::memcpy.

```
void printFloatBits(float value) {
    uint32_t bits;
    std::memcpy(&bits, &value, sizeof(value));
    // Далее bits можно сдвигать и маскировать.
}
```

Функция std::memcpy копирует объектное представление байт в байт. Числовое преобразование при этом не выполняется. Например, static_cast<uint32_t>(5.5) дал бы целое число 5, тогда как std::memcpy переносит в uint32_t именно 32 бита, которыми закодировано значение 5.5.

Для float используется uint32_t, а для double - uint64_t, поскольку в исследуемой системе размеры этих типов совпадают соответственно с 32 и 64 битами. После копирования над целочисленным представлением можно выполнять сдвиги и накладывать маски, что позволяет выделить знак, экспоненту и мантиссу.

```
std::cout << ((bits >> 31) & 1u);       // знак float
std::cout << ((bits >> i) & 1u);        // отдельный бит
поля
```

```
std::cout << ((bits >> 63) & 1ull);    // знак double
```

Тот же подход используется при изменении отдельных битов float и double: сначала представление копируется в беззнаковый целочисленный тип, затем изменяется маской, после чего новая последовательность битов копируется обратно в исходный вещественный тип.

```
std::memcpy(&bits, &value, sizeof(value));
bits ^= (1u << numberBit);
std::memcpy(&value, &bits, sizeof(value));
```

5. Битовые операции, применённые в работе

Для работы с отдельными разрядами использовалась маска 1 << k, где k - номер бита. Сдвиг переносит единственную единицу в требуемую позицию. Далее применяются логические побитовые операции:

- **Установка бита:** `value |= (1u << k)` - операция OR принудительно делает выбранный разряд равным 1.
- **Сброс бита:** `value &= ~(1u << k)` - инвертированная маска содержит 0 только в выбранной позиции, поэтому AND сбрасывает этот бит.
- **Инверсия бита:** `value ^= (1u << k)` - XOR с единицей меняет 0 на 1 и 1 на 0.

Если выбранный бит уже находится в требуемом состоянии, значение не изменится. Например, сброс бита, который уже равен 0, оставляет исходное число без изменений. Это наблюдается при работе с вещественными значениями и является нормальным результатом побитовой операции.

6. Сдвиг как арифметическая операция

В индивидуальном задании было реализовано умножение целого числа на 2 только через битовый сдвиг. Сдвиг всех битов на одну позицию влево увеличивает вес каждого установленного разряда в два раза, поэтому для значения, помещающегося в используемый тип, операция соответствует умножению на 2.

```
int multiplyByTwoWithShift(int value) {
    unsigned bits = static_cast<unsigned>(value);
    bits <<= 1;
    return static_cast<int>(bits);
}
```

Например, 5 имеет двоичное представление 00000101_2. После сдвига влево на один разряд получается 00001010_2, то есть 10. Аналогично, сдвиг влево на n позиций соответствует умножению на 2^n при условии отсутствия переполнения.

7. Различия моделей представления данных в памяти

Главное различие между целочисленной и плавающей моделями состоит в смысле одних и тех же битов. В целочисленном типе разряды непосредственно формируют двоичное значение (для знакового типа - в дополнительном коде). В IEEE 754 последовательность разделена на логические поля, и изменение одного бита может менять знак, порядок или дробную часть числа.

У float старший бит 31 отвечает за знак, биты 30-23 образуют экспоненту, а биты 22-0 - мантиссу. У double аналогичные поля занимают бит 63, биты 62-52 и 51-0.

Следует также различать порядок битов в логическом числе и порядок байтов в памяти. Понятие endianness относится к расположению байтов. На распространённых архитектурах x86/x86-64 используется little-endian: младший байт многобайтового значения располагается по меньшему адресу. При этом std::memcpy не переставляет байты, а копирует их в том порядке, в котором они находятся в объекте.

8. Выводы

В ходе работы были изучены два принципиально разных способа кодирования числовых данных. Целые знаковые числа представлены в дополнительном коде, который удобен для выполнения арифметики на уровне процессора. Вещественные значения представлены по IEEE 754 и состоят из

знака, экспоненты и мантиссы. Формат дополнительно поддерживает денормализованные значения, бесконечности и NaN.

Для исследования битового представления float и double был использован std::memcpy. Этот способ не выполняет числового преобразования, а переносит объектное представление байт в байт в целочисленный тип подходящего размера. После этого с помощью сдвигов и масок можно безопасно анализировать или изменять отдельные биты.

Практические эксперименты с установкой, сбросом и инверсией разрядов показали, что одинаковые побитовые операции имеют разный смысл для разных типов данных: у int они непосредственно изменяют двоичное целое значение, а у float и double могут затрагивать знак, экспоненту или мантиссу. Это демонстрирует, что интерпретация последовательности битов определяется типом данных и выбранной моделью представления.